Claude CCDV-F Cheat Sheet

APPLICATIONS AND INTEGRATION
33.1% OF EXAM

Applications Skill Weights · Messages API Core · Stop Reasons · HTTP Error Codes · Streaming Events · Message Batches API · API Feature Picker

MODEL SELECTION AND OPTIMIZATION
16.8% OF EXAM

Model and Prompt Weights · Model Tiers · Prompt Caching · Rate Limits · Model Tier Picker

AGENTS AND WORKFLOWS
14.7% OF EXAM

Agent and Tool Weights · Workflow Patterns · Agent Loop Anatomy · Claude Agent SDK · Agent or Workflow

PROMPT AND CONTEXT ENGINEERING
11.0% OF EXAM

Prompt Design · Context Management · Output Handling · System vs User Prompt

TOOLS AND MCPS
10.6% OF EXAM

Tool Definition · MCP Essentials · Tool Types · Tool Approach Picker

SECURITY AND SAFETY
8.1% OF EXAM

Security Skill Weights · Security Controls · Keys and Secrets · Prompt Rule vs Hook

CLAUDE CODE
3.1% OF EXAM

Claude Code Config · Permission Modes · CLAUDE.md hierarchy · settings.json precedence

EVAL, TESTING, AND DEBUGGING
2.6% OF EXAM

Debugging Method · Eval and Regression · Retry Rules · Trace analysis

QUICK FACTS

EXAM CODE CCDV-F	CREDENTIAL Claude Certified Developer Foundations
ITEMS 53 multiple-choice/multiple-response	TIME 120 minutes
PASS 720 of 100-1,000	FEE $125 USD
DOMAINS 8 weighted domains	DELIVERY Pearson VUE proctored
VALIDITY 12 months	RETAKE WAITS 14, 30, 90 days
ATTEMPTS Four per rolling year	BLUEPRINT v1.0, July 2026

WEIGHT ORDER

Apps 33, Model 17, Agents 15

Prompt and context 11 · Tools and MCP 10.6 · Security 8.1 · Code 3.1, Eval 2.6

STREAMING VS BATCH

Streaming	Batch
Tokens arrive live	Submit and poll
Someone is waiting	Nobody waiting
Full price	Half price

Now vs overnight

API FEATURE PICKER

Bulk job, nobody waiting	→	**Message Batches API** (50% cost)
User watching output	→	**Streaming** (SSE deltas)
Very large max_tokens	→	**Streaming** (Avoids timeout)
Same large prefix repeats	→	**Prompt caching** (0.1x reads)
Response must be JSON	→	**Structured outputs**
Claude must call code	→	**Custom tool** (input_schema)
Same file used repeatedly	→	**Files API**
Need cost estimate first	→	**count_tokens**
History outgrows window	→	**Compaction** (Or clearing)

APPLICATIONS SKILL WEIGHTS

Claude Application Design
8.6%
largest skill

Software Engineering Foundations
7.4%

Claude API Mechanics
6.8%

Configuration Management
4.1%

Understanding Requirements
3.4%

Systems Life Cycle
2.8%

STREAM EVENT ORDER

Start -> Blocks -> Deltas -> Stop

message_start opens · content_block_delta repeats · message_delta has stop_reason · Token counts are cumulative

MESSAGES API CORE

model
Exact model ID

max_tokens
Output ceiling, required

system
Top-level standing instructions

messages
user and assistant turns

tools
Tool schema array

stream
Incremental SSE output

Stateless
Resend history every call

usage
Token counts returned

request-id
Header for support tracing

STOP REASONS

end_turn
Finished naturally

max_tokens
Truncated at your ceiling

tool_use
Tool call pending

stop_sequence
Custom sequence hit

pause_turn
Resumable long turn

refusal
Declined; read stop_details

HTTP ERROR CODES

AP

400
invalid_request_error

401
authentication_error

402
billing_error

403
permission_error

404
not_found_error

409
conflict_error

413
request_too_large

429
rate_limit_error
retry

500
api_error
retry

504
timeout_error

529
overloaded_error
retry

STREAMING EVENTS

AP

message_start
Empty message shell

content_block_start
Block opens

content_block_delta
Incremental chunk

content_block_stop
Block closes

message_delta
stop_reason and usage

message_stop
Stream ends

ping
Keepalive, safely ignored

text_delta
Text fragment

input_json_delta
Partial tool arguments

thinking_delta
Reasoning fragment

signature_delta
Precedes thinking block stop

MESSAGE BATCHES API

AP

Discount
50% of standard prices

Typical run
Most finish within 1 hour

Hard window
Expires after 24 hours

Batch cap
100,000 requests or 256MB

custom_id
Matches result to request

Ordering
Results return in any order

Retention
Results available 29 days

processing_status
in_progress then ended

Result types
succeeded errored canceled expired

max_tokens
Must be at least 1

REQUEST SIZE LIMITS

AP

Messages API
32 MB

Token Counting API
32 MB

Batch API
256 MB

Files API
500 MB

Over the limit
413 request_too_large

CACHE PREFIX ORDER

MO

Tools -> System -> Messages

Stable content first · Volatile content last · Four breakpoints maximum · Any change invalidates

PROMPT CACHING VS BATCHES

MO

Prompt caching	Batches
Repeated prefix	Unique requests
Realtime latency	Async, 24 hours
0.1x cache reads	50% price

Reuse prefix vs wait

MODEL TIER PICKER

MO

Hardest reasoning work	→	Opus (Capability first)
Balanced production default	→	Sonnet
High volume, simple task	→	Haiku (Cheapest)
Latency is the constraint	→	Haiku
Cost too high	→	Cache, then lower effort
Quality slipped	→	Raise effort first
Behavior changed on deploy	→	Pin the model ID
Two options look equal	→	Run an eval

MODEL AND PROMPT WEIGHTS

MO

Technical Fundamentals
6.1%

LLM Fundamentals
5.2%

Prompt Engineering
4.6%

Context Engineering
3.8%

Cost and Token Management
2.8%

Model Selection Tradeoffs
2.7%

Output Handling
2.6%

MODEL TIERS

MO

Opus
Complex agentic, enterprise tier

Sonnet
Balanced speed and intelligence

Haiku
Fastest and cheapest tier

Pinned model ID
Upgrades become deliberate

effort
low medium high xhigh max

Adaptive thinking
Model decides reasoning depth

Fast mode
Higher throughput, premium price

Real metric
Cost per completed task

PROMPT CACHING

MO

Prefix order
tools, system, messages

Breakpoints
Four maximum per request

Default TTL
5 minutes

Extended TTL
1 hour option

Write cost
1.25x base input

Read cost
0.1x base input

cache_read_input_tokens
Proves a cache hit

Invalidator
Any prefix byte change

Placement
Stable first, volatile last

RATE LIMITS

MO

RPM
Requests per minute

ITPM
Input tokens per minute

OTPM
Output tokens per minute

Algorithm
Token bucket, continuously replenished

Cache reads
Usually excluded from ITPM

max_tokens
Does not affect OTPM

retry-after
Seconds to wait

Scope
Per organization, per model

TOKEN AND COST LEVERS

MO

count_tokens
Size prompt before sending

Prompt caching
First free saving

Batch API
Half price, asynchronous

Lower effort
Fewer reasoning tokens

Smaller model
Only if quality holds

Prune tool output
Shrinks resent history

Streaming
Avoids idle-connection timeouts

Context window
Shared by every part

AGENT SDK VS CLAUDE CODE

AG

Agent SDK	Claude Code
Library you embed	Terminal and IDE
Python or TypeScript	Interactive session
Programmatic agent	Human in loop

Embed vs operate

AGENT OR WORKFLOW

AG

One call would work	→	Single LLM call (Start here)
Fixed ordered subtasks	→	Prompt chaining
Distinct input categories	→	Routing
Independent parallel subtasks	→	Sectioning
Need higher confidence	→	Voting
Subtasks unknown upfront	→	Orchestrator-workers
Clear grading criteria	→	Evaluator-optimizer
Steps cannot be scripted	→	Autonomous agent
Context filling up	→	Subagent isolation

AGENT AND TOOL WEIGHTS

AG

Agent Construction
5.3%

Agent Patterns
4.9%

Agent Architecture
4.5%

Tool Implementation
4.4%

Agentic Customization
4.1%

MCP Server Development
2.1%

WORKFLOW VS AGENT

AG

Workflow	Agent
Predefined code paths	Claude directs steps
Predictable	Open-ended
Easy to test	Harder to predict

You route vs Claude routes

WORKFLOW PATTERNS

AG

Single call
Default starting point

Prompt chaining
Fixed ordered subtasks

Routing
Classify, then specialize

Parallel sectioning
Independent subtasks merged

Parallel voting
Same task repeated

Orchestrator-workers
Runtime subtask decomposition

Evaluator-optimizer
Draft and critique loop

Subagent
Isolated context window

AGENT LOOP ANATOMY

AG

Request
Model plus tool definitions

stop_reason
Loop continuation signal

tool_use
Execute, then return result

tool_result
Sent in user turn

end_turn
Loop terminates

Hook
Deterministic enforced checkpoint

Iteration cap
Stops runaway loops

CLAUDE AGENT SDK

AG

Languages
Python and TypeScript

Built-in tools
Read, write, edit, bash

Agent loop
Supplied, not hand-written

Subagents
Focused sub-tasks

Hooks
Agent lifecycle code points

Permissions
Auto-run versus approval

Sessions
Resume or fork context

MCP
External tool connection

Hosting
Runs in your process

SYSTEM VS USER PROMPT

PR

System	User
Role and standing rules	Task and data
Output format	Changes every turn
Stable across turns	Carries untrusted input

Durable frame vs task

PROMPT DESIGN

PR

System prompt
Role and standing rules

User turn
Task and its data

XML-style tags
Mark section boundaries

Few-shot
Three to five examples

Long documents
Place near the top

Output constraints
State the format explicitly

Zero-shot
Instruction, no examples

Iteration
Refine against real failures

CLEARING VS COMPACTION

PR

Clearing	Compaction
Drops tool results	Summarizes history
Leaves a placeholder	Keeps a trace
Cheap and predictable	Costs a pass

Delete vs summarize

CONTEXT MANAGEMENT

PR

Context window
System, tools, history, output

Clearing
Drops old tool results

Compaction
Summarizes earlier history

Context drift
Instructions lost downstream

Tool output pruning
Cuts accumulated bloat

Subagent isolation
Separate context budget

Thinking blocks
Return them unchanged

STRUCTURED OUTPUT VS STRICT

PR

Structured outputs	Strict tool use
Shapes the response	Shapes tool arguments
Schema you supply	Per tool definition
What Claude says	How Claude calls

Response vs arguments

OUTPUT HANDLING

PR

Structured outputs
Response matches your schema

Strict tool use
Arguments match your schema

Defensive parsing
Validate before consuming

Field checks
Presence, type, range

Confident tone
Not evidence of correctness

Non-determinism
Judge across several runs

Fallback path
Handle parse failure

TOOL ROUND TRIP

TO

Use -> Run -> Result -> Reply

stop_reason is tool_use · Execute your handler · tool_result in user turn · Same tool_use_id

TOOL USE VS MCP

TO

Tool use	MCP server
Schema in your app	Separate running server
Serves one application	Serves many applications
You execute calls	Standard protocol

In-app vs shared server

TOOL APPROACH PICKER

TO

Search web or run code	→ Server tool (No handler)
Reach your own system	→ Custom tool
Reuse across many apps	→ MCP server
Adding procedure, not capability	→ Skill (No server)
Need file editing	→ Anthropic-schema tool
Arguments must validate	→ strict: true
Handler threw an error	→ tool_result is_error
Several calls at once	→ One user message

TOOL DEFINITION

TO

name
Unique tool identifier

description
Primary selection signal

input_schema
JSON Schema parameters

strict
Guarantees schema-valid arguments

tool_use
Claude's call block

tool_result
Your returned output

tool_use_id
Pairs call with result

is_error
Flags a handler failure

SKILL VS MCP SERVER

TO

Skill	MCP server
Instructions on demand	New callable functions
Uses existing tools	Backed by a system
No server to run	Process to deploy

Procedure vs capability

MCP ESSENTIALS

TO

Tools
Executable server actions

Resources
Contextual data sources

Prompts
Reusable interaction templates

Elicitation
Server asks the user

Protocol
JSON-RPC 2.0 messages

stdio
Local process transport

Streamable HTTP
Remote server transport

Host
Application running the clients

Client
One per connected server

TOOL TYPES

TO

Custom tool
You define and execute

Server tool
Anthropic runs it

Anthropic-schema tool
Bash, text editor, memory

Skill
Procedure loaded on demand

MCP server
Reusable across applications

Default choice
Least code you maintain

INJECTION DEFENSE

SE

Isolate, restrict, enforce with hooks

Untrusted in own block · Minimum tool scope · Hook blocks the action · Polite prompt rules fail

PROMPT RULE VS HOOK

SE

Prompt rule	Hook
Advisory guidance	Runs every time
Model may deviate	Can block the call
No enforcement	Deterministic control

Guidance vs enforcement

SECURITY SKILL WEIGHTS

SE

AI Application Security
3.2%

Claude Code Operation
3.1%

Debugging and Error Handling
2.6%

Guardrails and Safe Deployment
2.3%

Identity, Secrets, Keys
1.6%

Claude Hooks
1.0%

smallest skill

SECURITY CONTROLS

SE

Prompt injection
Untrusted text read as instruction

Isolation
Untrusted content, own block

Least privilege
Minimum tool scope

Hooks
Enforced, not advisory

PII
Redact before the request

Retrieval scoping
Only this user's data

Output filtering
Screen before display

Guardrail layering
Defense in depth

KEYS AND SECRETS

SE

Storage
Secrets manager, server side

Client code
Never ship a key

Repository
Never commit a key

Rotation
Replace on a schedule

Expiration
Set a key lifetime

Suspected leak
Disable the key immediately

Call origin
From your own backend

CLAUDE CODE CONFIG

CL

Managed policy
Org-wide, cannot be excluded

User memory
~/.claude/CLAUDE.md

Project memory
./CLAUDE.md, checked in

Local memory
CLAUDE.local.md, gitignored

Load behavior
Concatenated, never overriding

Read order
Broadest scope first

Imports
@path, five hops deep

settings.json
Behavior, permissions, hooks

PERMISSION MODES

CL

default
Prompts on first use

acceptEdits
Auto-accepts file edits

plan
Explores without editing

auto
Classifier-checked auto-approval

bypassPermissions
Skips permission prompts

Rule order
deny, then ask, allow

PreToolUse hook
Blocks before the call

RETRY RULES

EV

Retry 429, 500s, 529; never 4xx

Exponential backoff · Honor retry-after · SDKs retry twice · 400 401 403 404 permanent

DEBUGGING METHOD

EV

Step one
Classify the error type

Integration fault
Handler or arguments wrong

Model fault
Wrong output, correct data

Trace
Read the tool sequence

request-id
Ties failure to request

Recovery
Retry, fallback, or fail

Reproduce
Sample runs, not one

EVAL AND REGRESSION

EV

Eval set
Cases with expected behavior

Quality gate
Run before shipping changes

Model swap
Behavior change, needs eval

Prompt edit
Same risk, no compiler

Exact-match test
Will flake

Versioned artifacts
Prompts, schemas, model ID

Comparison
New config versus current

Monitoring
Watch quality in production

COMMON TRAPS

Workflow vs agent
Workflow follows your code ≠ Agent chooses its steps

Batch vs parallel calls
Batches cut price 50% ≠ Parallel calls pay full price

Custom tool vs MCP
Tool lives in one app ≠ MCP server serves many

Prompt rule vs hook
Prompt rules only advise ≠ Hooks actually block

Result matching
Match batches by custom_id ≠ Never match by position

Cache read vs write
Cache reads cost 0.1x ≠ Cache writes cost 1.25x

Thinking block handling
Return them unchanged ≠ Editing them causes 400

Skill vs MCP server
Skill adds procedure ≠ MCP adds capability

tool_result placement
Goes in a user message ≠ Not the assistant turn

Domain score vs pass
Total scaled score decides ≠ Domain percentages are informational

Structured output vs strict
Structured shapes the response ≠ Strict shapes tool arguments

Bank size vs exam
Real exam has 53 items ≠ Practice banks are larger

LAST MINUTE

1. 53 items, 120 minutes, 720/1000
2. Applications 33.1% is one third
3. Model selection 16.8%, agents 14.7%
4. Cache order: tools, system, messages
5. Batches = 50% cost, 24 hours
6. Match batch results by custom_id
7. Messages API is stateless; resend history
8. tool_result rides in user message
9. Same tool_use_id pairs call, result
10. Failed handler = tool_result is_error
11. Retry 429, 500s, 529; never 4xx
12. MCP = tools, resources, prompts
13. stdio is local; streamable HTTP remote
14. Hooks enforce; prompt rules only advise
15. Untrusted content in its own block
16. System = rules; user = task
17. CLAUDE.md files concatenate, never override
18. Pin the model ID in production
19. Eval before every prompt change
20. Retake waits: 14, 30, 90 days

Independent exam preparation notice

OpenExamPrep is an independent education provider, not affiliated with or endorsed by the exam sponsors named on this site. Exam and certification names identify what our resources help you study for; they do not imply sponsorship, approval, or review of our materials. Unless a resource expressly identifies authorized public material, our practice questions are independently developed simulations, not actual, recalled, or sponsor-provided exam items. We aim to reflect publicly available exam outlines and topic information in our study resources. Coverage, format, and difficulty may differ from the actual exam, and we cannot guarantee that every detail is accurate or current. Confirm exam requirements, fees, and policies with the official exam sponsor.

Updated 2026-09-05. Official exam sources checked July 2026 (Exam Guide v1.0). This is a study aid only and is not for use during the exam.